

Universidad Europea de Madrid

Laboratorio de Hacking

Practica 2 - Reconocimiento activo

Autor/a: Adrian Garcia Mejias

Titulacion: Grado en Ingenieria Informatica

Asignatura: Laboratorio de Hacking

Fecha de entrega: 26 de abril

Resumen

Este trabajo pone a prueba, de forma controlada y sobre un pequeño laboratorio propio (no contra ningún sistema real ajeno), dos técnicas básicas para averiguar que máquinas están encendidas en una red y que «puertas» (puertos) tienen abiertas para ofrecer servicios.

En la primera parte se ha escrito un pequeño programa que envía distintos tipos de «llamadas» a un grupo de direcciones de red – parecido a llamar a varias puertas de un edificio para ver cuáles responden – y que distingue con precisión cuáles de esas máquinas están realmente encendidas y cuáles no. El programa se ha probado contra dos máquinas virtuales controladas por el propio autor, confirmando que detecta correctamente ambas como activas, y que ignora el resto de direcciones donde no hay ninguna máquina.

En la segunda parte se ha estudiado que hace exactamente la herramienta `nmap` cuando se usa sin ninguna configuración especial, algo que mucha gente da por hecho sin comprobar. Se ha confirmado que, por defecto, `nmap` solo comprueba las 1000 «puertas» más habituales de una máquina, lo que significa que un servicio menos común – como ocurrió con una base de datos Redis en este laboratorio – puede pasar completamente desapercibido si no se le indica expresamente donde buscar. Además, se ha capturado el tráfico de red generado durante estas pruebas para comprobar, paquete a paquete, exactamente como reconoce la herramienta si una puerta está abierta o cerrada.

Los resultados confirman una idea central para cualquier persona responsable de la seguridad de una red: las herramientas de escaneo, por muy fiables que parezcan, tienen comportamientos por defecto que pueden dejar cosas fuera de la vista si no se conocen a fondo, y conviene siempre verificar lo que realmente está pasando en la red antes de confiar ciegamente en el resultado de una única herramienta.

Indice

Resumen	2
1 Introduccion	4
2 Desarrollo	4
2.1 Descubrimiento de hosts con scapy	4
2.1.1 Entorno de pruebas	5
2.1.2 Resultados de la ejecucion	5
2.1.3 Diseno y modularidad del codigo	6
2.2 Comportamiento por defecto de nmap y estado de puertos	7
2.2.1 Estado de un puerto: definicion y estímulos	7
2.2.2 Comportamiento por defecto de nmap	8
3 Resultados	10
3.1 Descubrimiento de hosts	10
3.2 Comportamiento por defecto de nmap	11
4 Conclusiones	11
5 Bibliografia	12

1 Introduccion

A diferencia de la Practica 1, centrada exclusivamente en tecnicas **pasivas** de recopilacion de informacion, esta segunda practica aborda el **reconocimiento activo**: el envio deliberado de estímulos de red (paquetes ICMP, TCP, UDP) contra un objetivo y el analisis de sus respuestas, con el fin de determinar que hosts estan activos y que puertos/servicios exponen. A diferencia del reconocimiento pasivo, esta interaccion es directamente observable por el objetivo (y por cualquier sistema de deteccion de intrusiones que este vigilando su red).

Por este motivo, y tal como exige expresamente el enunciado de la asignatura, **todas** las pruebas de esta practica se han realizado exclusivamente contra un entorno de laboratorio propio, simulado mediante contenedores **Docker** en una red aislada, sin dirigir en ningun momento trafico de reconocimiento activo hacia sistemas ajenos ni expuestos a Internet.

La practica se divide en dos bloques:

1. **Descubrimiento de hosts con scapy**: implementacion de una funcion en Python (`craft_discovery_pkts`) que construye paquetes de sondeo ICMP (timestamp), TCP (ACK) y UDP para determinar, sin depender de herramientas de terceros, que direcciones IP de un rango dado corresponden a hosts activos.
2. **Comportamiento por defecto de nmap**: analisis en profundidad, apoyado en captura de trafico con `tcpdump`, del comportamiento que adopta `nmap` cuando se ejecuta sin opciones de configuracion explicitas – cuantos paquetes envia, contra cuantos puertos, y que patrones de respuesta permiten distinguir un puerto abierto de uno cerrado.

Los objetivos concretos de esta practica son:

1. Definir una funcion de crafeo de paquetes multi-protocolo, con argumentos obligatorios y opcionales bien gestionados.
2. Emplear dicha funcion para distinguir hosts activos de inactivos en un entorno controlado.
3. Definir el concepto de estado de puerto (abierto, cerrado, filtrado) y los estímulos/respuestas que permiten determinarlo, tanto en TCP como en UDP.
4. Identificar el comportamiento por defecto de `nmap` en el reconocimiento de puertos abiertos, con ejemplos reales.
5. Justificar dichos hallazgos mediante evidencia de captura de trafico a nivel de paquete.

En ningun momento de esta practica se ha dirigido trafico de reconocimiento activo hacia sistemas que no fueran contenedores propios, desplegados y controlados en su totalidad por el autor de este informe.

2 Desarrollo

2.1 Descubrimiento de hosts con scapy

Para evaluar el descubrimiento de hosts mediante estímulos activos se ha implementado la funcion `craft_discovery_pkts`, que construye paquetes scapy de tres tipos: ICMP (Timestamp Request, tipo 13), TCP con el flag ACK activado sin completar el saludo de tres vias, y UDP. La eleccion de ICMP Timestamp en lugar del clasico Echo Request responde al enunciado de la practica y constituye una alternativa menos habitual y potencialmente menos filtrada que el ping tradicional. El envio de un segmento TCP con unicamente el flag ACK es una tecnica de descubrimiento pasivo-agresiva bien conocida: un host activo respondera con un paquete RST independientemente de si el puerto esta abierto o cerrado, ya que el paquete no pertenece a

ninguna conexion establecida, lo que permite confirmar actividad sin necesidad de completar ninguna conexion real.

La funcion admite hasta tres protocolos (lista o cadena unica), un rango de IPs en formato CIDR, rango con guion o IP unica, un diccionario opcional con el numero de paquetes a construir por protocolo, y un puerto opcional para las capas TCP/UDP (por defecto, 80).

2.1.1 Entorno de pruebas

Todas las pruebas se han realizado exclusivamente contra un laboratorio propio y aislado, construido con **Docker** [1]: una red bridge dedicada (**labnet**, 172.20.0.0/24) con dos contenedores en IPs fijas (172.20.0.10 con **nginx** y 172.20.0.11 con **Redis**), y el resto del rango sin ningun servicio en ejecucion, empleado como referencia de host inactivo. En ningun momento se ha dirigido trafico hacia sistemas ajenos al propio laboratorio.

2.1.2 Resultados de la ejecucion

Se invoco la funcion contra el rango completo 172.20.0.10-172.20.0.60 (51 direcciones), combinando los tres protocolos con distintos conteos de paquete (ICMP: 1, TCP: 2, UDP: 1), enviando un total de **204 paquetes** mediante `sr()` de `scapy` [2]. Del total, se recibieron **2 respuestas**, correspondientes exactamente a las direcciones de los dos contenedores activos (172.20.0.10 y 172.20.0.11); el resto del rango, sin ningun servicio en ejecucion, no genero ninguna respuesta (ver Figura 1). Este resultado confirma que la funcion distingue correctamente hosts activos de inactivos usando estímulos de las tres capas de transporte solicitadas.

```

WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: more MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: more MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: more MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: more MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: more MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: more MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: more MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: more MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: MAC address to reach destination not found. Using broadcast.
WARNING: more MAC address to reach destination not found. Using broadcast.
^CWARNING: MAC address to reach destination not found. Using broadcast.
[*] Paquetes enviados: 204
[*] Respuestas recibidas: 2
[*] Sin respuesta: 31

IPs activas detectadas:
- 172.20.0.10
- 172.20.0.11

(adr3d0@adr3d0)-[~/Laboratorio1-hacking-uem/P2_Reconocimiento_activo/src]
$ sudo $(which uv) run python3 host_discovery.py

```

Figura 1: Ejecucion de `host_discovery.py`: 204 paquetes enviados (ICMP/TCP/UDP), 2 respuestas correspondientes a los hosts activos del laboratorio Docker.

2.1.3 Diseño y modularidad del código

La implementación separa deliberadamente dos responsabilidades en dos ficheros distintos: `craft_discovery_pkts.py` contiene únicamente la construcción de paquetes (sin efectos secundarios, no envía nada), y `host_discovery.py` se encarga de la lógica de envío, recepción y presentación de resultados. Esta separación facilita la reutilización de `craft_discovery_pkts` en otros contextos (por ejemplo, para construir paquetes con fines distintos al descubrimiento de hosts) sin arrastrar dependencias de red o de impresión por pantalla.

La expansión de rangos de IP (`_expand_ip_range`) soporta tres formatos de entrada (CIDR, rango con guion entre dos IPs completas, o IP única) para maximizar la flexibilidad de uso sin imponer una única sintaxis al usuario de la función.

```

def craft_discovery_pkts(
    protocols: str | list[str],
    ip_range: str,

```

```

    packet_counts: dict[str, int] | None = None,
    port: int = 80,
) -> list:
    if isinstance(protocols, str):
        protocols = [protocols]
    protocols = [p.upper() for p in protocols]

    if len(protocols) > 3:
        raise ValueError("Se admiten como maximo 3 protocolos.")
    invalid = set(protocols) - VALID_PROTOCOLS
    if invalid:
        raise ValueError(f"Protocolo(s) no soportado(s): {invalid}")

    if packet_counts is None:
        packet_counts = {proto: 1 for proto in protocols}

    ips = _expand_ip_range(ip_range)

    packets = []
    for proto in protocols:
        n = packet_counts.get(proto, 1)
        for ip in ips:
            for _ in range(n):
                if proto == "ICMP":
                    pkt = IP(dst=ip) / ICMP(type=13)
                elif proto == "TCP":
                    pkt = IP(dst=ip) / TCP(dport=port, flags="A", sport=RandShort())
                else:
                    pkt = IP(dst=ip) / UDP(dport=port, sport=RandShort())
                packets.append(pkt)
    return packets

```

El código fuente completo, con la lógica de expansión de rangos y el script de envío, se encuentra en P2_Reconocimiento_activo/src/ de este mismo repositorio.

2.2 Comportamiento por defecto de nmap y estado de puertos

2.2.1 Estado de un puerto: definicion y estímulos

El **estado de un puerto** es la clasificación que una herramienta de reconocimiento asigna a un puerto de un host tras enviarle un estímulo y observar (o no observar) una respuesta. Nmap distingue principalmente tres estados para TCP:

Abierto existe un proceso escuchando activamente en ese puerto. Ante un estímulo SYN, el host responde con SYN/ACK, confirmando su disposición a completar el establecimiento de la conexión.

Cerrado el host está activo y responde, pero ningún proceso escucha en ese puerto. Ante un SYN, el host responde con RST/ACK, rechazando la conexión de forma explícita.

Filtrado un firewall o dispositivo intermedio bloquea el tráfico, por lo que no se puede determinar el estado real del puerto. Se manifiesta como ausencia total de respuesta, o como un mensaje ICMP tipo 3 (Destination Unreachable) con código indicativo de bloqueo administrativo.

Para UDP, al carecer de un mecanismo de confirmación de conexión como el de TCP, la determinación del estado es menos precisa:

Abierto o abierto|filtrado si el puerto UDP esta realmente abierto y la aplicacion no responde nada (comportamiento habitual salvo que se envíe una carga util especifica del protocolo), nmap no puede distinguirlo de un puerto filtrado, y lo marca como **open|filtered**.

Cerrado el host responde con un mensaje ICMP tipo 3, codigo 3 (Port Unreachable), confirmando de forma inequivoca que no hay ningun proceso escuchando ahi.

Filtrado se recibe un ICMP tipo 3 con codigos 1, 2, 9, 10 o 13, indicativos de un bloqueo administrativo (firewall) en lugar de la simple ausencia de un servicio.

2.2.2 Comportamiento por defecto de nmap

Para observar el comportamiento por defecto de nmap se ha capturado el trafico generado durante tres ejecuciones con **tcpdump** sobre la interfaz del laboratorio Docker, empleando las opciones **-Pn** (omitir el descubrimiento de hosts) y **-n** (omitir la resolucion de nombres), tal y como exige el enunciado para centrar el analisis unicamente en el reconocimiento de puertos [3], [4].

La primera ejecucion, **nmap -Pn -n 172.20.0.10**, identifico el puerto 80 como abierto (servicio **http**, correspondiente al contenedor **nginx**) y los 999 puertos restantes como cerrados. El analisis del trafico capturado confirma que nmap, ejecutado como root, emplea por defecto un **TCP SYN scan (-sS)**: un unico paquete SYN por puerto, con el mismo puerto origen fijo para todos los envios, contra exactamente **1000 puertos** – la lista de puertos mas frecuentes segun la base de datos **nmap-services** – lo que se ha verificado contando directamente los paquetes SYN sin ACK dirigidos al host, resultando en **1000 paquetes** enviados (ver Figura 2).

Una segunda ejecucion contra **172.20.0.11** (el contenedor **Redis**) reporto los 1000 puertos por defecto como cerrados, pese a que el servicio Redis escucha realmente en el puerto 6379. Al forzar explicitamente ese puerto con **nmap -Pn -n -p 6379 172.20.0.11**, se confirma que el servicio si esta abierto. Este resultado ilustra una limitacion relevante del comportamiento por defecto de nmap: al limitar el escaneo a los 1000 puertos estadisticamente mas comunes, servicios legitimos pero menos habituales (como Redis) pueden pasar completamente desapercibidos si no se amplia el rango de puertos o se especifican explicitamente (**-p-** para los 65535, o **-p <puerto>** para uno concreto).


```

libEGL warning: egl: failed to create dri2 screen

(adr3d0@adr3d0)-[~/Laboratorio1-hacking-uem/P2_Reconocimiento_activo/doc]
$ nmap -Pn -n 172.20.0.10
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-06 14:27 +0100
Nmap scan report for 172.20.0.10
Host is up (0.0000010s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: 02:42:AC:14:00:0A (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.15 seconds

(adr3d0@adr3d0)-[~/Laboratorio1-hacking-uem/P2_Reconocimiento_activo/doc]
$ nmap -Pn -n 172.20.0.11
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-06 14:28 +0100
Nmap scan report for 172.20.0.11
Host is up (0.0000010s latency).
All 1000 scanned ports on 172.20.0.11 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 02:42:AC:14:00:0B (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds

(adr3d0@adr3d0)-[~/Laboratorio1-hacking-uem/P2_Reconocimiento_activo/doc]
$ nmap -Pn -n -p 6379 172.20.0.11
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-06 14:28 +0100
Nmap scan report for 172.20.0.11
Host is up (0.000038s latency).

PORT      STATE SERVICE
6379/tcp  open  redis
MAC Address: 02:42:AC:14:00:0B (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds

(adr3d0@adr3d0)-[~/Laboratorio1-hacking-uem/P2_Reconocimiento_activo/doc]
$ 

```

Figura 2: Ejecucion de nmap por defecto contra los dos contenedores del laboratorio: puerto 80 abierto en 172.20.0.10, y confirmacion de que el puerto 6379 de 172.20.0.11 queda fuera del escaneo por defecto.

El analisis a nivel de paquete con tcpdump sobre el fichero de captura confirma dos patrones claramente diferenciados segun el estado del puerto (ver Figura 3):

- **Puerto abierto (80/tcp):** intercambio de 3 paquetes – SYN del origen, SYN/ACK de respuesta del host, y un RST final del origen para cerrar la conexion sin completarla, caracteristico del **half-open scan** que da nombre al TCP SYN scan.
- **Puerto cerrado (22/tcp, sin ningun servicio SSH en el contenedor):** intercambio de solo 2 paquetes – SYN del origen y RST/ACK inmediato del host, rechazando la conexion sin necesidad de mas intercambios.

```

14:27:59.401648 IP 172.20.0.1.44997 > 172.20.0.10.21: Flags [S], seq 150358082
8, win 1024, options [mss 1460], length 0
14:27:59.401656 IP 172.20.0.1.44997 > 172.20.0.10.993: Flags [S], seq 15035808
28, win 1024, options [mss 1460], length 0
14:27:59.401659 IP 172.20.0.1.44997 > 172.20.0.10.135: Flags [S], seq 15035808
28, win 1024, options [mss 1460], length 0
14:27:59.401662 IP 172.20.0.1.44997 > 172.20.0.10.554: Flags [S], seq 15035808
28, win 1024, options [mss 1460], length 0
14:27:59.401665 IP 172.20.0.1.44997 > 172.20.0.10.443: Flags [S], seq 15035808
28, win 1024, options [mss 1460], length 0
tcpdump: Unable to write output: Broken pipe

(adr3d0@adr3d0)-[~/Laboratorio1-hacking-uem/P2_Reconocimiento_activo/doc]
$ tcpdump -r /tmp/nmap_capture.pcap -n "src host 172.20.0.1 and dst host 172
.20.0.10 and tcp[tcpflags] & tcp-syn != 0 and tcp[tcpflags] & tcp-ack == 0" 2>
/dev/null | wc -l
1000

(adr3d0@adr3d0)-[~/Laboratorio1-hacking-uem/P2_Reconocimiento_activo/doc]
$ tcpdump -r /tmp/nmap_capture.pcap -n "host 172.20.0.10 and port 80"
reading from file /tmp/nmap_capture.pcap, link-type EN10MB (Ethernet), snapsho
t length 262144
14:27:59.401696 IP 172.20.0.1.44997 > 172.20.0.10.80: Flags [S], seq 150358082
8, win 1024, options [mss 1460], length 0
14:27:59.401721 IP 172.20.0.10.80 > 172.20.0.1.44997: Flags [S.], seq 90594272
2, ack 1503580829, win 64240, options [mss 1460], length 0
14:27:59.401723 IP 172.20.0.1.44997 > 172.20.0.10.80: Flags [R], seq 150358082
9, win 0, length 0

(adr3d0@adr3d0)-[~/Laboratorio1-hacking-uem/P2_Reconocimiento_activo/doc]
$ tcpdump -r /tmp/nmap_capture.pcap -n "host 172.20.0.10 and port 22"
reading from file /tmp/nmap_capture.pcap, link-type EN10MB (Ethernet), snapsho
t length 262144
14:27:59.401631 IP 172.20.0.1.44997 > 172.20.0.10.22: Flags [S], seq 150358082
8, win 1024, options [mss 1460], length 0
14:27:59.401632 IP 172.20.0.10.22 > 172.20.0.1.44997: Flags [R.], seq 0, ack 1
503580829, win 0, length 0

(adr3d0@adr3d0)-[~/Laboratorio1-hacking-uem/P2_Reconocimiento_activo/doc]
$ 

```

Figura 3: Captura con tcpdump de los patrones de paquetes para un puerto abierto (80, 3 paquetes) y un puerto cerrado (22, 2 paquetes).

3 Resultados

A continuacion se resumen de forma tabulada los principales resultados de la investigacion.

3.1 Descubrimiento de hosts

Protocolo	Paquetes/IP	Puerto (TCP/UDP)	Estimulo
ICMP	1	N/A	Timestamp Request (tipo 13)
TCP	2	80 (por defecto)	Flag ACK, sin conexion establecida
UDP	1	80 (por defecto)	Datagrama vacio

Tabla 1: Configuracion empleada en la invocacion de `craft_discovery_pkts` contra el rango 172.20.0.10-172.20.0.60 [2].

Metrica	Valor	Interpretacion
IPs en el rango	51	172.20.0.10 a 172.20.0.60
Paquetes enviados	204	51 IPs $\times$ 4 paquetes (1 ICMP + 2 TCP + 1 UDP)
Respuestas recibidas	2	Corresponden exactamente a los 2 contenedores activos
IPs activas detectadas	172.20.0.10, 172.20.0.11	nginx y Redis, respectivamente

Tabla 2: Resultados de la ejecucion de `host_discovery.py`.

3.2 Comportamiento por defecto de nmap

Host	Puerto	Servicio	Estado (por defecto)
172.20.0.10	80	http (nginx)	Abierto
172.20.0.10	22	ssh	Cerrado
172.20.0.11	6379	redis	No escaneado por defecto (fuera del top 1000)
172.20.0.11	6379 (forzado con -p)	redis	Abierto

Tabla 3: Resultados de los escaneos `nmap -Pn -n` contra el laboratorio, incluyendo el caso de un servicio fuera del rango de puertos por defecto [3].

Estado	Paquetes intercambiados	Secuencia de flags
Abierto	3	SYN -> SYN/ACK -> RST
Cerrado	2	SYN -> RST/ACK

Tabla 4: Patrones de trafico observados con `tcpdump` segun el estado del puerto, en TCP SYN scan [4].

4 Conclusiones

Esta practica ha demostrado, de forma controlada y sobre un laboratorio propio, como el reconocimiento activo permite pasar de una simple sospecha de actividad a una confirmacion tecnica precisa: que hosts estan vivos, que puertos tienen abiertos, y que herramientas revelan ese estado mediante el analisis directo de los paquetes intercambiados.

La implementacion de `craft_discovery_pkts` confirma que es posible construir, con muy pocas lineas de codigo y sin depender de herramientas externas, un mecanismo de descubrimiento de hosts multi-protocolo (ICMP, TCP, UDP) capaz de distinguir con precision hosts activos de inactivos, incluso empleando tecnicas menos habituales que el ping clasico (como el ICMP Timestamp) o sin completar ninguna conexion real (TCP ACK sin handshake).

El analisis del comportamiento por defecto de `nmap` deja dos aprendizajes centrales. En primer lugar, que su configuracion por defecto (TCP SYN scan contra los 1000 puertos mas comunes) es extremadamente eficiente – un unico paquete SYN por puerto, sin necesidad de completar ninguna conexion – pero **no exhaustiva**: el caso de Redis en el puerto 6379, invisible en el escaneo por defecto y detectado unicamente al forzar el puerto de forma explicita, demuestra que confiar ciegamente en la configuracion por defecto de una herramienta puede dejar fuera servicios reales y potencialmente relevantes desde el punto de vista de seguridad. En segundo lugar, que el packet sniffing (`tcpdump`) es una herramienta imprescindible para **verificar** el

comportamiento real de una herramienta de mas alto nivel, en lugar de asumir su documentacion o su salida por pantalla sin contrastarla a nivel de paquete.

Desde una perspectiva de seguridad ofensiva, estas dos observaciones se traducen en recomendaciones practicas: (1) al auditar una red propia o autorizada, ampliar el rango de puertos analizado (-p-) cuando el objetivo pueda alojar servicios no convencionales, y (2) validar siempre mediante captura de trafico el comportamiento exacto de las herramientas empleadas, en particular cuando su resultado condicione decisiones de seguridad posteriores. Desde una perspectiva defensiva, la leccion es simetrica: un servicio expuesto en un puerto no habitual no es «invisible» de forma fiable frente a un atacante decidido, solo frente a un escaneo superficial con la configuracion por defecto.

5 Bibliografia

- [1] Docker Inc., «Docker – Container platform». [En línea]. Disponible en: <https://www.docker.com/>
- [2] Biondi, Philippe and contributors, «Scapy – Python packet manipulation library». [En línea]. Disponible en: <https://scapy.net/>
- [3] Lyon, Gordon "Fyodor", «Nmap – Network Mapper». [En línea]. Disponible en: <https://nmap.org/>
- [4] The Tcpdump Group, «tcpdump – Command-line packet analyzer». [En línea]. Disponible en: <https://www.tcpdump.org/>